
PORTFOLIO - GRADUAAT PROGRAMMEREN

Persoonlijk Dashboard

Een webapplicatie met vier API-gedreven widgets

De ontwerper · Doel 6: Je documenteert de oplossing

Auteur Jonas Trap

Opleiding Graduaat Programmeren, Thomas More Geel

Academiejaar 2025-2026

Type bewijs eigen project

Het project

Dit dashboard is een persoonlijk project: een single-page webapplicatie die vier dagelijkse noden samenbrengt op één centrale pagina. De vier widgets tonen het actuele weerbericht voor opgeslagen locaties, een dagelijks avondmaalrecept met filters voor dieet en bereidingstijd, de NASA-foto van de dag met uitleg, en een eenvoudig takenbeheer.

Het project dient als demonstratiestuk van mijn vaardigheden in webontwikkeling en API-integratie. De nadruk lag bewust op een nette, professionele uitwerking die de geleerde concepten concreet toepast: een duidelijke architectuur, type-veiligheid, integratie met externe API's en degelijke foutafhandeling.

Technische keuzes

- **Build tool:** Vite, voor een snelle ontwikkelomgeving met native TypeScript-ondersteuning.
- **Taal:** TypeScript, voor type-veiligheid en betere onderhoudbaarheid.
- **CSS-framework:** Bootstrap 5, voor een degelijke grid en ingebouwde dark mode.
- **API's:** OpenWeatherMap (weer), Spoonacular (recepten), NASA APOD (foto van de dag), en een vertaal-API voor de Nederlandse inhoud.

Broncode

De volledige broncode van dit project staat publiek op GitHub, zodat ze altijd terug te vinden is: github.com/jonastapmore/jonastap-projecten/tree/main/dashboard

De oplossing documenteren

Ik koppel dit project aan dit doel omdat ik de oplossing volledig heb gedocumenteerd in een uitgebreide README, aangevuld met commentaar in de code zelf. De README is bewust geschreven zodat een lezer zonder de code te openen begrijpt wat het project wil bereiken en wat het precies doet. Hieronder de twee kernpunten van het doel.

1. De doelen van de ontwikkeling documenteren

De README opent met een duidelijke **doelstelling** en een **projectplanning** in fasen, elk met een eigen doel en status. Zo is gedocumenteerd waarom het project bestaat en wat elke ontwikkelstap moest opleveren.

```
## Doelstelling
```

```
Het doel van dit project is het bouwen van een persoonlijk dashboard als webapplicatie. Dit dashboard brengt vier dagelijkse noden samen op een centrale pagina: het actuele weerbericht, een dagelijks avondmaalrecept met filters, de NASA-foto van de dag, en een eenvoudig takenbeheer.
```

```
### Fase 3: Widget Recept
```

- Integratie met Spoonacular
- Willekeurig recept ophalen en weergeven
- Filterlogica implementeren voor Keto, Low Calorie en Snel klaar
- Ingrediëntenlijst, voedingswaarde-badges en de vertaalhulp verwerken

```
**Status:** Afgerond
```

Fragmenten uit de README: de doelstelling en een fase uit de planning.

2. De scope en functionaliteiten vastleggen

Voor elke widget legt de README de **scope en functionaliteiten** vast: wat de widget toont, welke API hij gebruikt en wat hij wel en niet doet. Een aparte voortgangstabel en een API-overzicht maken de afbakening compleet.

```
### Widget 2: Menu van de Dag
```

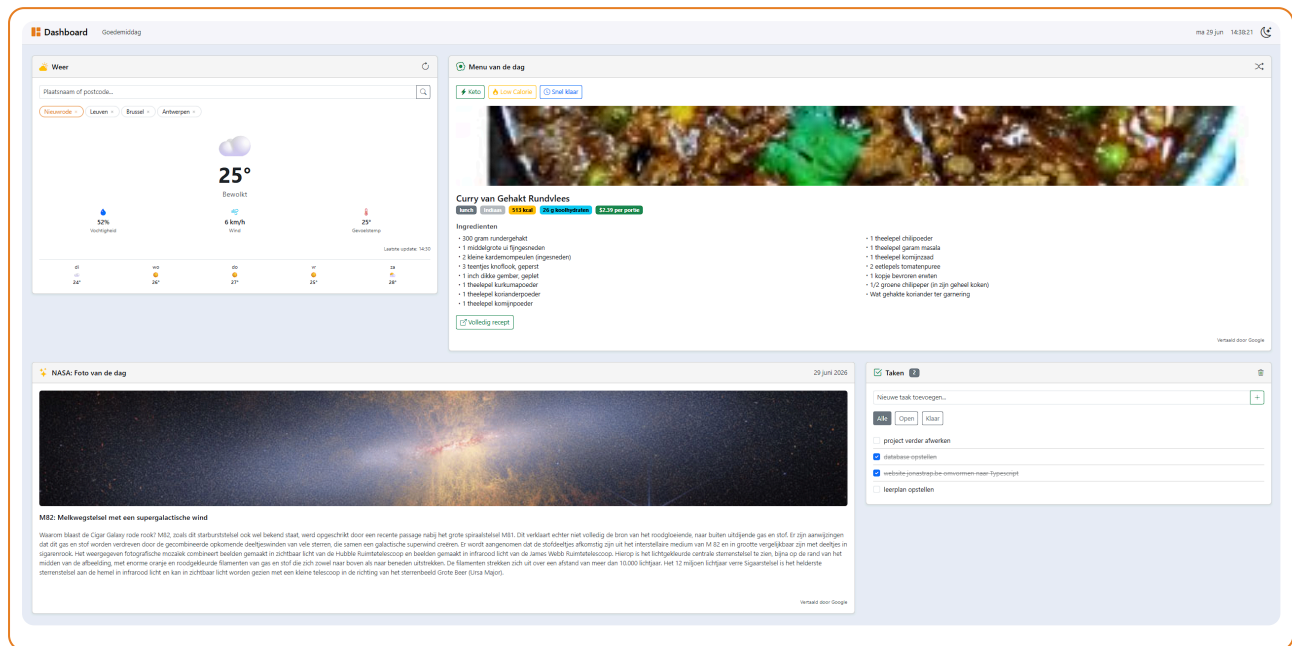
```
**Functionaliteit:**
```

- Toont een willekeurig recept bij het laden van de pagina
- Filteropties: Keto, Low Calorie en Snel klaar (binnen 30 minuten)
- Ingrediëntenlijst, vertaald naar het Nederlands
- Badges met calorieën, koolhydraten en de prijs per portie
- Nette foutmelding bij een bereikt aanvraaglimiet of geen passend recept

Het API-overzicht legt de externe afhankelijkheden en de scope per widget vast:

Widget	API	Gratis	Registratie
Weer	OpenWeatherMap	Ja (1.000/dag)	API-sleutel via openweathermap.org
Recept	Spoonacular	Ja (daglimiet)	API-sleutel via spoonacular.com
NASA	NASA APOD	Ja	DEMO_KEY of gratis sleutel
Taken	localStorage	n.v.t.	Geen

Het resultaat



Afbeelding 1: het dashboard. De werking ervan is in de README volledig gedocumenteerd.

Documentatie in de code

Naast de README is de oplossing ook in de code zelf gedocumenteerd. Niet-vanzelfsprekende keuzes krijgen commentaar dat de reden uitlegt, zodat een lezer de bedoeling meteen begrijpt:

```
// De app draait online in de submap /dashboard/, dus moeten de
// asset-paden daarnaar verwijzen (anders zoekt de browser ze op
// de domeinroot).
base: '/dashboard/',
```

Codefragment uit `vite.config.ts`: commentaar dat een keuze verantwoordt.

Waar de documentatie staat

- **README.md**: doelstelling, technische keuzes, een beschrijving per widget, het API-overzicht, de projectplanning in fasen, veiligheid en setup-instructies.
- **Commentaar in de code**: Nederlandse uitleg bij niet-vanzelfsprekende keuzes en stappen.
- **GitHub**: de README wordt automatisch getoond op de openbare repository, zodat de documentatie altijd vindbaar is.

Zelfreflectie

Wat ging goed

De README groeide mee met het project: bij elke afgewerkte widget vulde ik de voortgangstabel, de functionaliteiten en de planning aan. Daardoor bleef de documentatie actueel en kon ik op elk moment tonen wat af was en wat het deed.

Wat ik geleerd heb

Ik leerde dat documentatie meer is dan een handleiding achteraf. Door de doelen en de scope vooraf vast te leggen, hield ik zelf overzicht over wat er nog moest gebeuren. Goede documentatie stuurt dus ook de ontwikkeling, niet alleen het gebruik.

Uitdagingen

De grootste uitdaging was de juiste hoeveelheid detail kiezen: genoeg om de scope en functionaliteiten duidelijk vast te leggen, zonder de README onleesbaar te maken. Ik loste dit op met een vaste structuur per widget (functionaliteit en technische aanpak gescheiden) en samenvattende tabellen.

Wat beter kan

- Screenshots opnemen in de README zelf, zodat ook visueel duidelijk is hoe elke widget eruitziet.
- Documentatie van de code uitbreiden met korte beschrijvingen per functie in een vaste stijl (bijvoorbeeld JSDoc).

Conclusie

Dit project toont de twee kernpunten van het doel aan: ik **documenteer de doelen van de ontwikkeling** via de doelstelling en de gefaseerde planning, en ik **leg de scope en functionaliteiten vast** via een beschrijving per widget, een API-overzicht en een voortgangstabel, aangevuld met commentaar in de code zelf.